

Algoritmos de Aprendizado

- › Regra de Hebb
- › Perceptron
- › Delta Rule (Least Mean Square)
- › Multi-Layer Perceptron (Back Propagation)
- › Competitive Learning
- › Radial Basis Functions (RBFs)
- › **Echo State Neural Networks (ESN)**

Echo State Neural Networks (ESN)

- › Introdução
- › Arquitetura
- › Funcionamento Geral
- › Treinamento
- › Observações
- › Aplicações:
 - › Previsão de Séries Temporais
 - › Identificação de Sistemas

Echo State Neural Networks (ESN)

- › Introdução
- › Arquitetura
- › Funcionamento Geral
- › Treinamento
- › Observações
- › Aplicações:
 - › Previsão de Séries Temporais
 - › Identificação de Sistemas

Echo State Networks (ESNs): Introdução

- › Redes Neurais Recorrentes (*RNNs*) representam uma grande variedade de modelos computacionais que simulam com maior ou menor precisão o cérebro humano.
- › Pode desenvolver uma ativação dinâmica auto-sustentada ao longo da sua conexão recorrente mesmo quando não tem entrada
 - ⇒ *RNN* pode ser vista como um sistema dinâmico
- › Preserva em seu estado interno uma transformação não linear do histórico de entradas
 - ⇒ memória dinâmica

Echo State Networks (ESNs): Introdução

- › Existem basicamente duas classes de *RNNs*:
 - › A primeira classe é caracterizada por uma dinâmica estocástica de minimização de energia e conexões simétricas.
 - › *Hopfield Networks*, *Boltzmann Machines* e *Deep Belief Networks* são as mais conhecidas
 - › A maioria treinadas com esquemas de aprendizagem não supervisionado
 - › A segunda classe tem características dinâmicas determinísticas para a atualização
 - › Utilizam esquema treinamento supervisionado.

Echo State Networks (ESNs): Introdução

- › RNNs foram menos utilizadas devido à dificuldade de serem treinadas pelo método de gradiente decrescente que buscam reduzir o erro de treinamento iterativamente.
- › Algumas das deficiências dos algoritmos propostos são:
 - › A mudança gradual dos parâmetros da rede durante o aprendizado pode não garantir a convergência
 - › Custo computacionalmente alto.
 - › Difícil aprendizado de dependências que requerem memória de longo-prazo.
 - › Algoritmos avançados podem ser utilizados mas necessitam de parametrização mais complexa

Echo State Networks (ESNs): Introdução

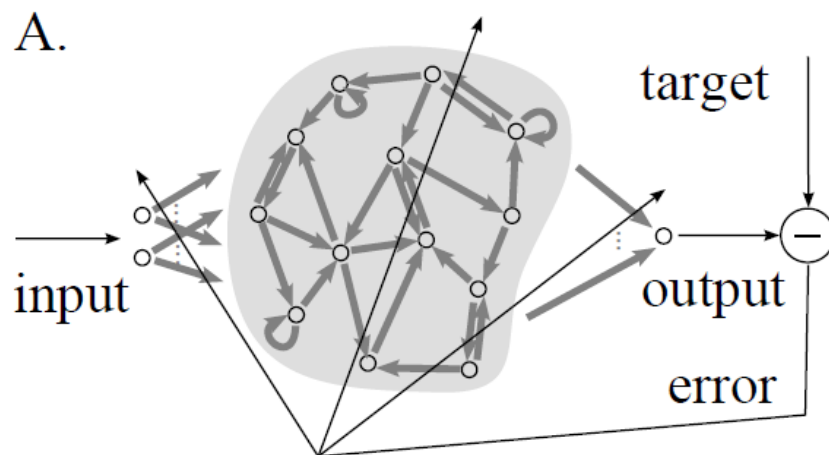
- › Em 2001, dois modelos independentes foram propostos por:
 - › Wolfgang Maass (***Liquid State Machines***) e
 - › Herbert Jaeger (***Echo State Networks***)

Também conhecida como **Reservoir Computing**.

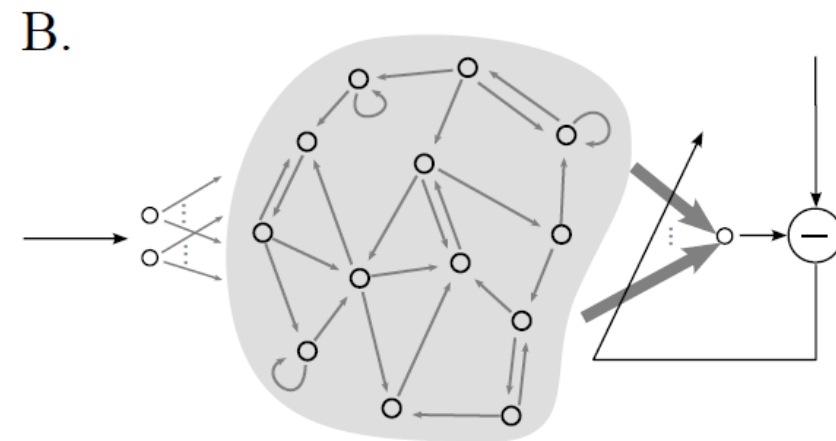
- › Jaeger propôs as **redes neurais** com **estados** de **eco** (em inglês, *Echo State Networks*, ESNs), as quais podem ser vistas como um promissor compromisso entre dois objetivos aparentemente conflitantes:
 - › (i) simplicidade do modelo matemático resultante; e
 - › (ii) capacidade de expressar uma ampla gama de comportamentos dinâmicos não-lineares.

ESNs: Introdução

- › A utilização de modelos ESNs evita problema de gradiente decrescente:
 - › Uma *RNN* é criada aleatoriamente e permanece fixa durante o treinamento (*Reservoir*), recebe os sinais de entrada e faz nos estados uma transformação não-linear do histórico de entrada.
 - › A saída desejada é gerada por uma combinação linear dos sinais dos neurônios do *Reservoir*; esta combinação linear é obtida por regressão linear utilizando o sinal *objetivo* como padrão.



A. Modelo tradicional de treinamento de RNN, adaptando todos os pesos das conexões de entrada, RNN interna e saída.



B. Modelo RC, somente os pesos da camada de saída são adaptados.

ESNs: Introdução

Plausibilidade / Capacidade do Modelo

- › RC é **computacionalmente viável** para sistemas de **tempo contínuo**, valores contínuos e **sistemas em tempo real** com **recursos limitados**.
- Comparações mostraram semelhanças entre os cérebros de mamíferos e as arquiteturas propostas pelos princípios de *Reservoir Computing*.

ESNs: Introdução

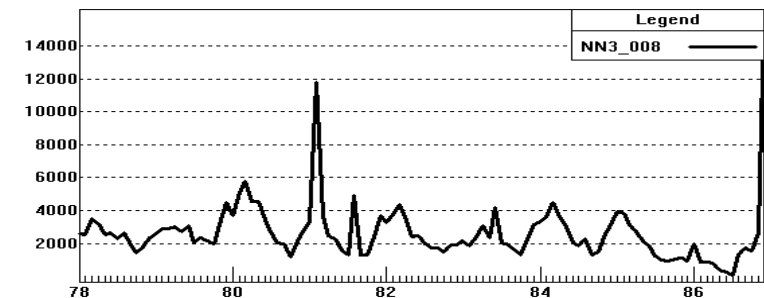
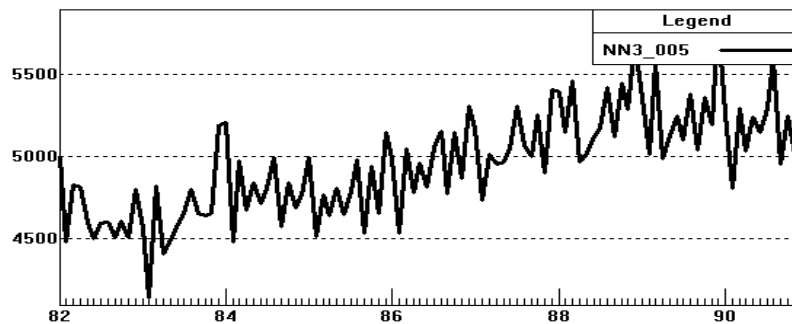
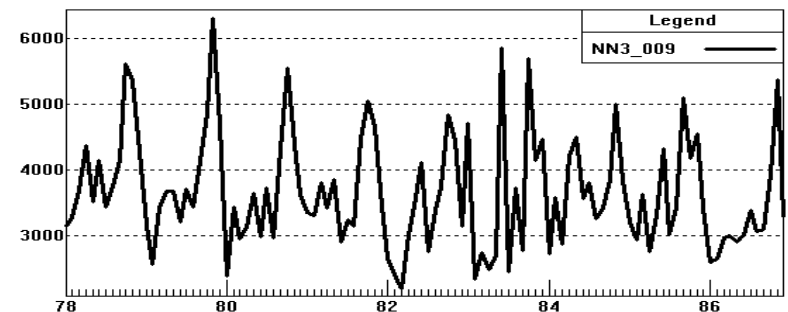
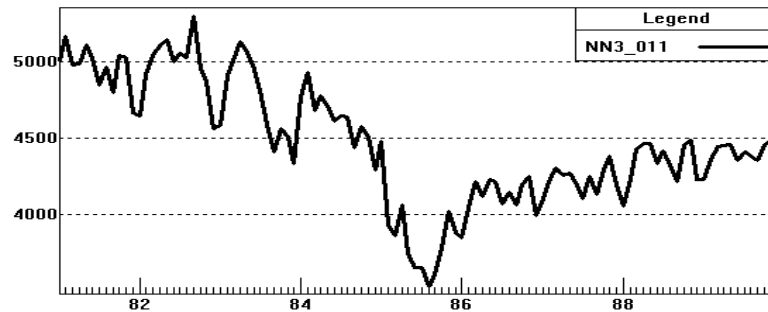
Precisão do Modelo

- › ESNs têm superado os métodos anteriores de sistemas de identificação **não-linear, predição e classificação**.
- › Em predição de **sistemas dinâmicos caóticos**:
 - › melhorou os resultados anteriores em três ordens de grandeza. (*Harnessing nonlinearity: predicting chaotic systems and saving energy in wireless communication*).
- › Em equalização de canais sem fio não-lineares:
 - › obteve resultados melhorando em duas ordens de grandeza os anteriores.
- › No “*Japanese Vowel Benchmark*” obteve um erro de 0% em testes, o erro do modelo anterior era de 1.8%.
- › Em reconhecimento de dígitos (áudio) obteve um resultados de 0% de taxa de erro.

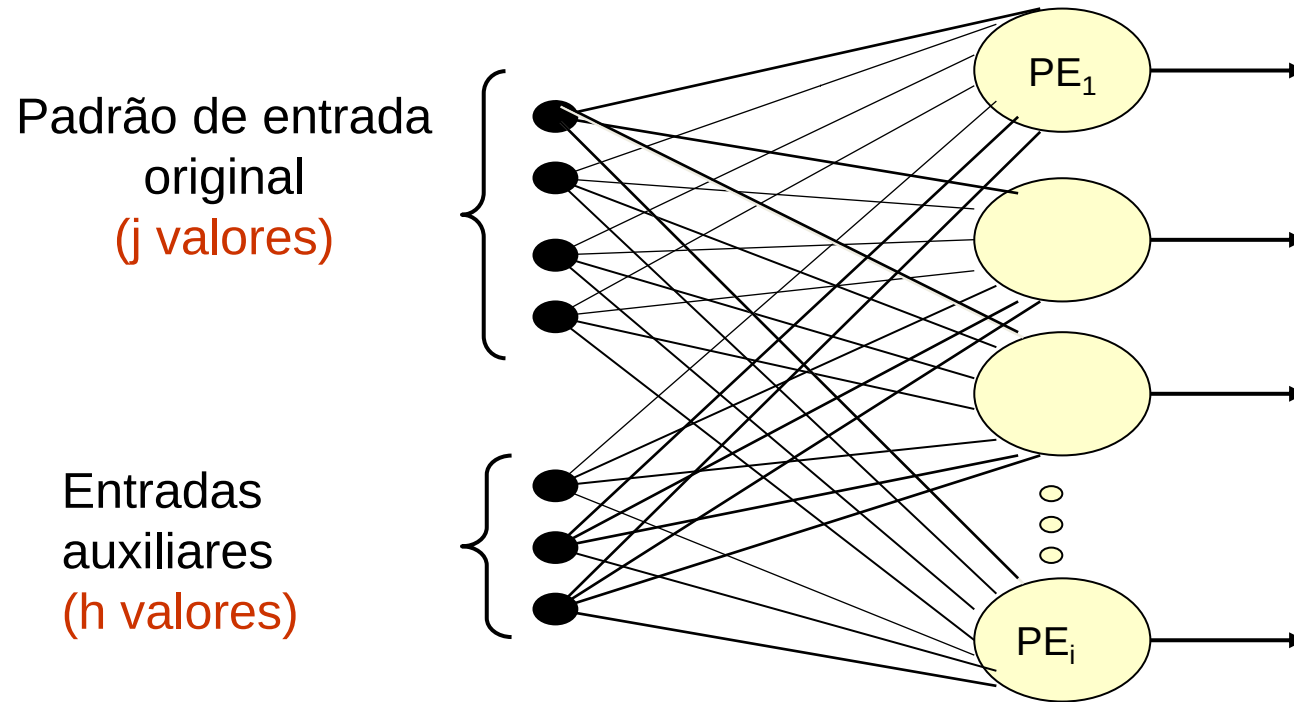
ESNs: Introdução

Precisão do Modelo

- › Em previsão financeira foi o ganhador da “*International Forecasting Competition NN3*”.



ESNs: Introdução - Functional Link Networks



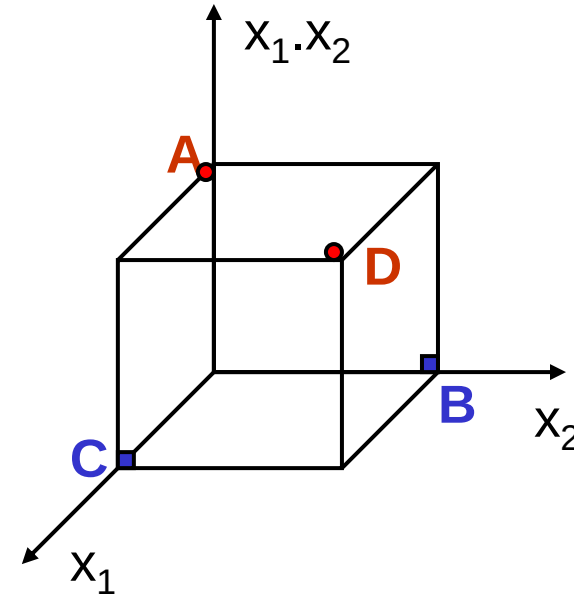
J	H	Entradas Adicionais	
2	1	$x_1 x_2$	-
3	4	$x_1 x_2, x_1 x_3, x_2 x_3$	$x_1 x_2 x_3$
4	10	$x_1 x_2, x_1 x_3, x_1 x_4, x_2 x_3, x_2 x_4, x_3 x_4$	$x_1 x_2 x_3, x_1 x_2 x_4, x_1 x_3 x_4, x_2 x_3 x_4$

ESNs: Introdução - Functional Link Networks

- **Exemplo do OU-EXCLUSIVO:**

- $J = 2$ (número de entradas originais - x_1, x_2)
- $H = 1$ (número de entradas adicionais - $x_1 \cdot x_2$)

Pontos	Entradas				Saída
A	-1	-1	1	$\Rightarrow$	-1
B	-1	1	-1	$\Rightarrow$	1
C	1	-1	-1	$\Rightarrow$	1
D	1	1	1	$\Rightarrow$	-1

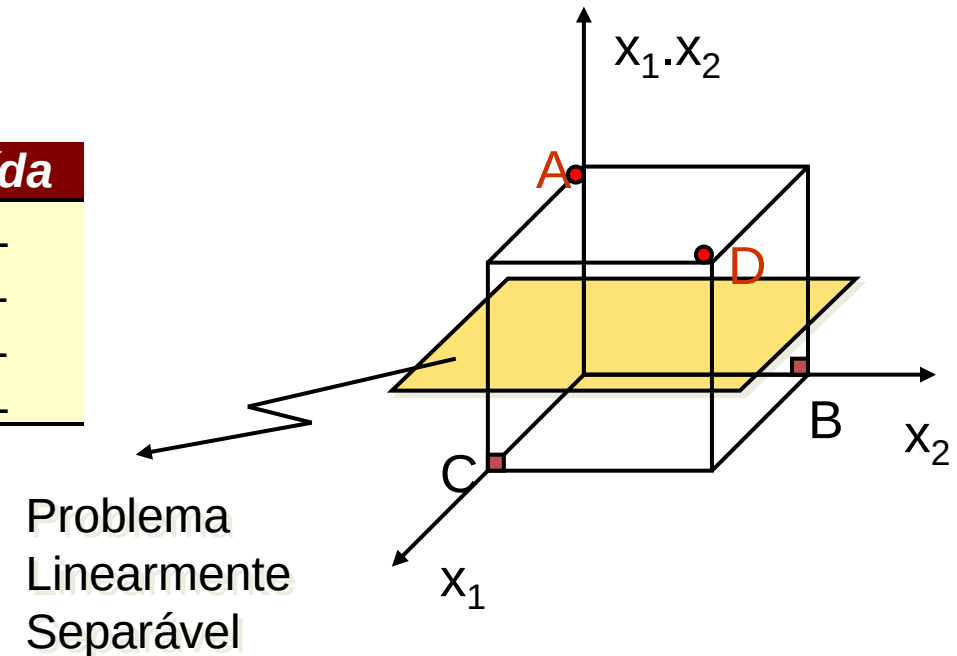


ESNs: Introdução - Functional Link Networks

- **Exemplo do OU-EXCLUSIVO:**

- $J = 2$ (número de entradas originais - x_1, x_2)
- $H = 1$ (número de entradas adicionais - $x_1 \cdot x_2$)

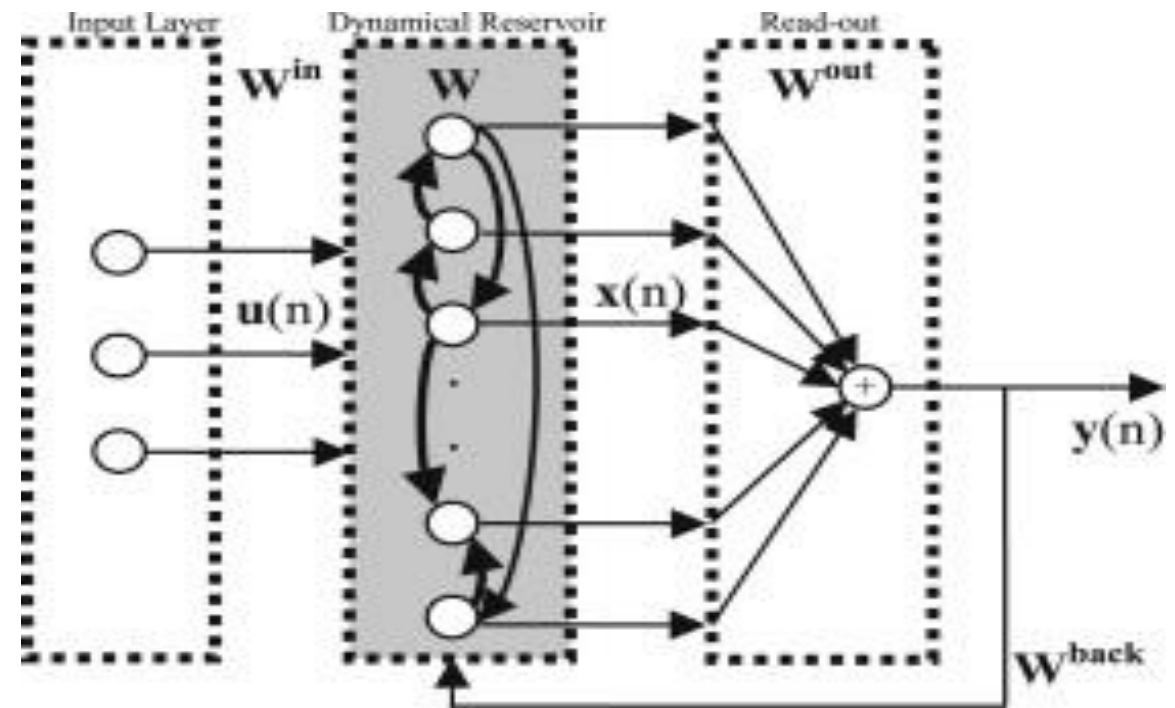
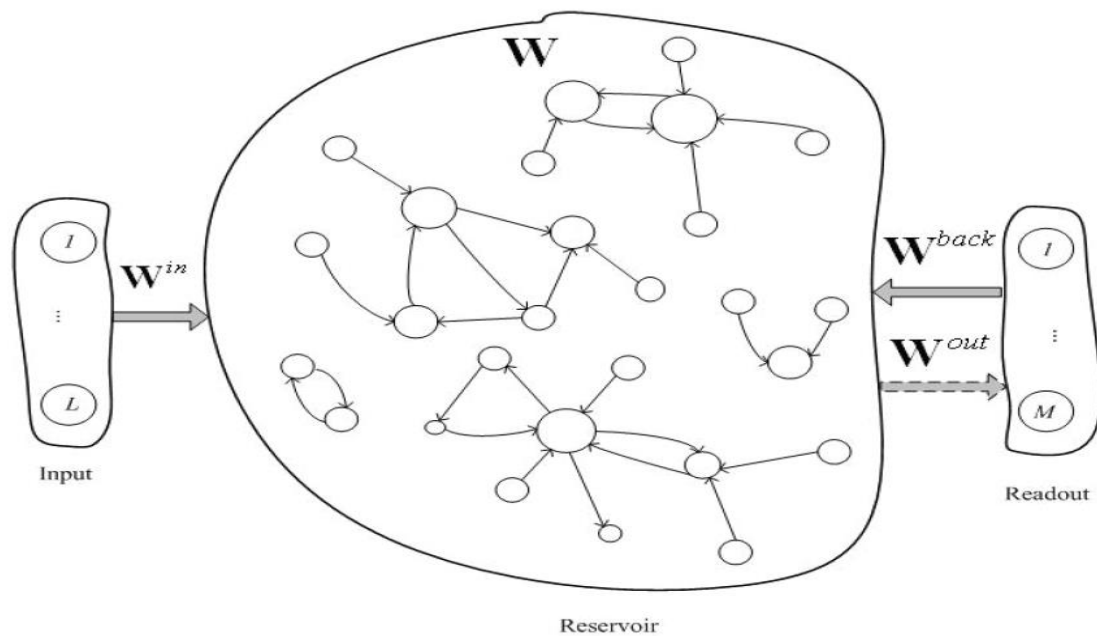
Pontos	Entradas				Saída
A	-1	-1	1	$\Rightarrow$	-1
B	-1	1	-1	$\Rightarrow$	1
C	1	-1	-1	$\Rightarrow$	1
D	1	1	1	$\Rightarrow$	-1



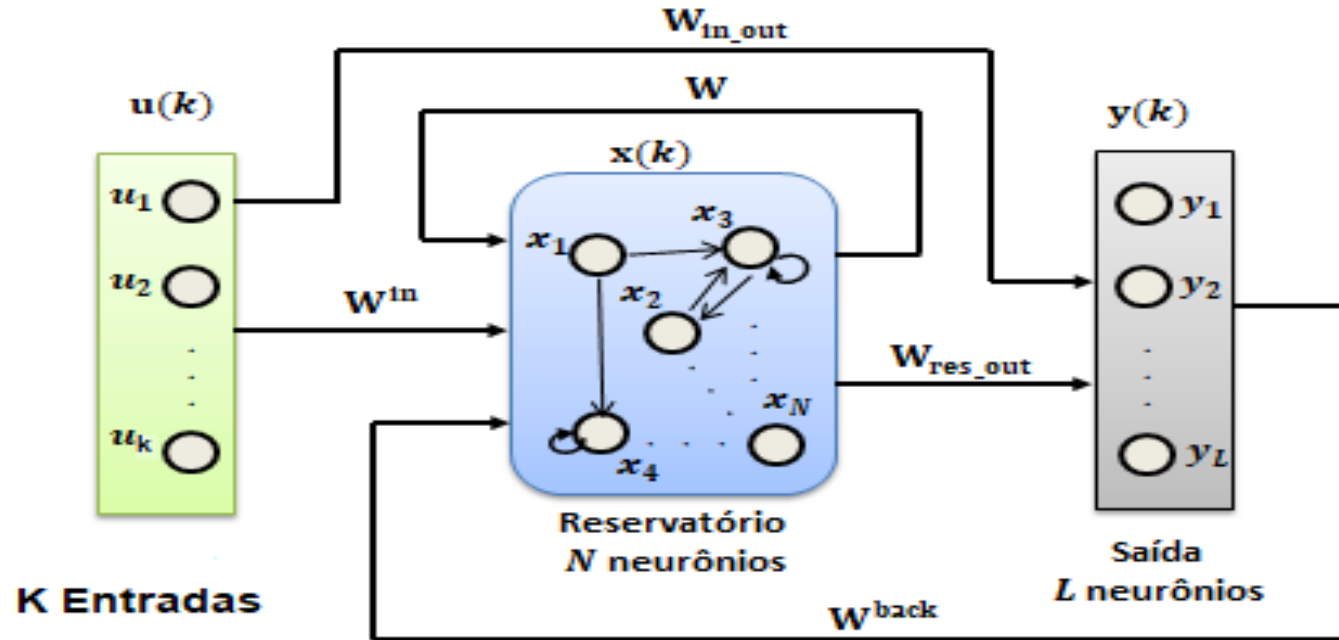
Echo State Neural Networks (ESN)

- › Introdução
- › **Arquitetura**
- › Funcionamento Geral
- › Treinamento
- › Observações
- › Aplicações:
 - › Previsão de Séries Temporais
 - › Identificação de Sistemas

ESNs: Arquitetura



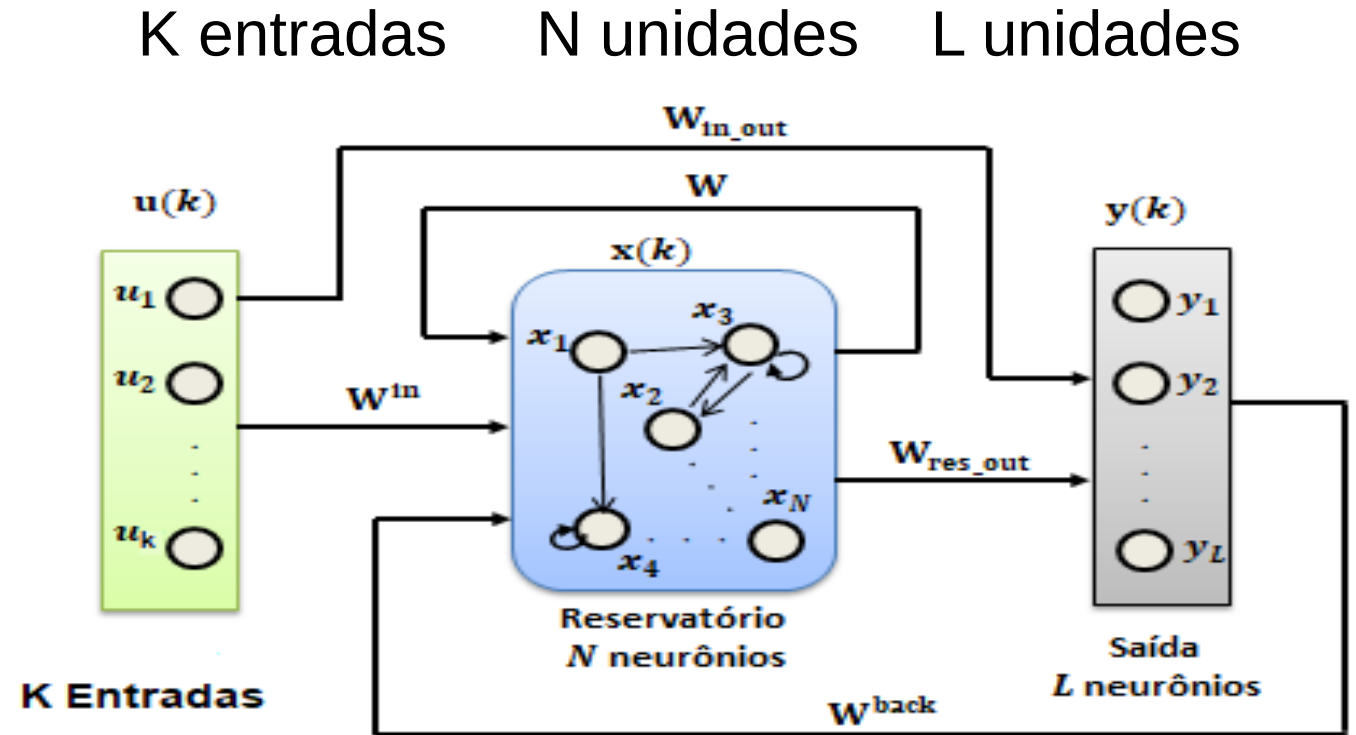
ESNs: Arquitetura



- Uma ESN é composta, basicamente, por:
 - uma camada de entrada,
 - um reservatório de neurônios arranjados de maneira recorrente, e
 - uma camada de saída.

ESNs: Arquitetura - Camadas

- › K: unidades de entrada
- › N: unidades internas da rede
- › L: unidades de saída
- › Em um tempo definido por n
 - › As entradas seriam:
 - › $u(n) = (u_1(n), \dots, u_K(n))$,
 - › As unidades internas seriam:
 - › $x(n) = (x_1(n), \dots, x_N(n))$,
 - › As unidades de saída seriam:
 - › $y(n) = (y_1(n), \dots, y_L(n))$



- › $x(\cdot)$ expande o histórico de entrada $u(n), u(n-1), \dots$ em um **espaço de estados complexo o suficiente** para que $y(\cdot)$ combine os sinais dos neurônios $x(n)$ na saída desejada $y_{target}(n)$ de forma linear.

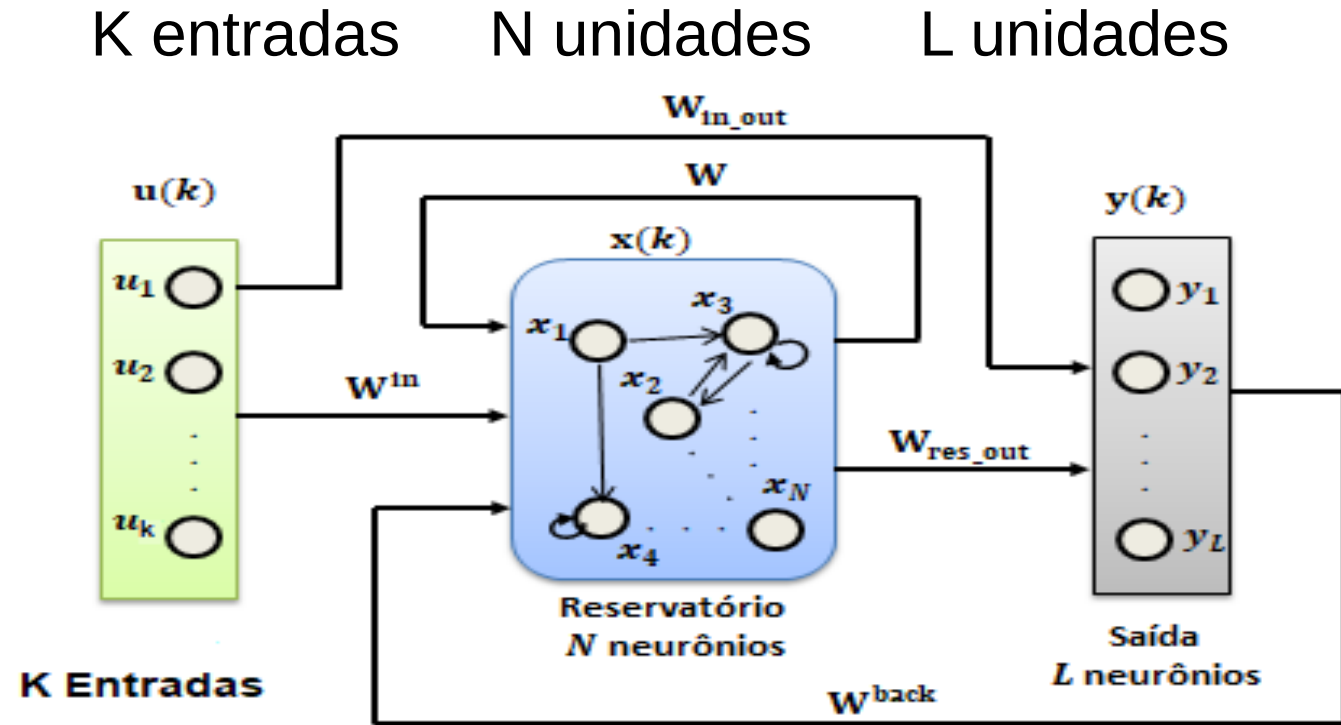
$$\mathbf{y}(n) = \mathbf{W}_{out}\mathbf{x}(n) = \mathbf{W}_{out}x(\mathbf{u}(n))$$

- › Para cada dimensão y_i de $\mathbf{y}$ existe um vetor de pesos de saída $(W_{out})_i$

$$(\mathbf{W}_{out})_i \mathbf{x}(n) = y_i(n) \approx y_{target,i}(n)$$

ESNs: Arquitetura - Matriz de Pesos

- › Matriz de pesos de entrada
 - › $K \times N \rightarrow W_{in}$
 - › $K \times L \rightarrow W_{in_out}$
- › Matriz do Reservoir (interna)
 - › $N \times N \rightarrow W$
- › Matriz de retorno
 - › $L \times N \rightarrow W_{back}$
- › Matriz de Saída
 - › $(K + N) \times L \rightarrow W_{out}$ ou W_{res_out}

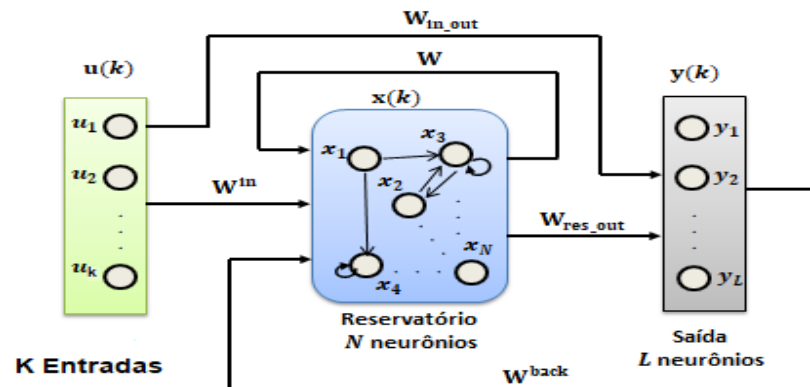


ESNs: Arquitetura - Matriz de Pesos

- › As **matrizes W , W_{in} e W_{back} permanecem fixas** após serem inicializadas aleatoriamente, enquanto que **W_{out} é a única matriz ajustada** durante o treinamento.
- › O reservatório possui **grande número de neurônios com conexões recorrentes, esparsas e inicializadas aleatoriamente.**

ESNs: Arquitetura - Reservoir

- O estado do 'reservoir' agrega o estado anterior do mesmo, as entradas da rede e a retroalimentação.



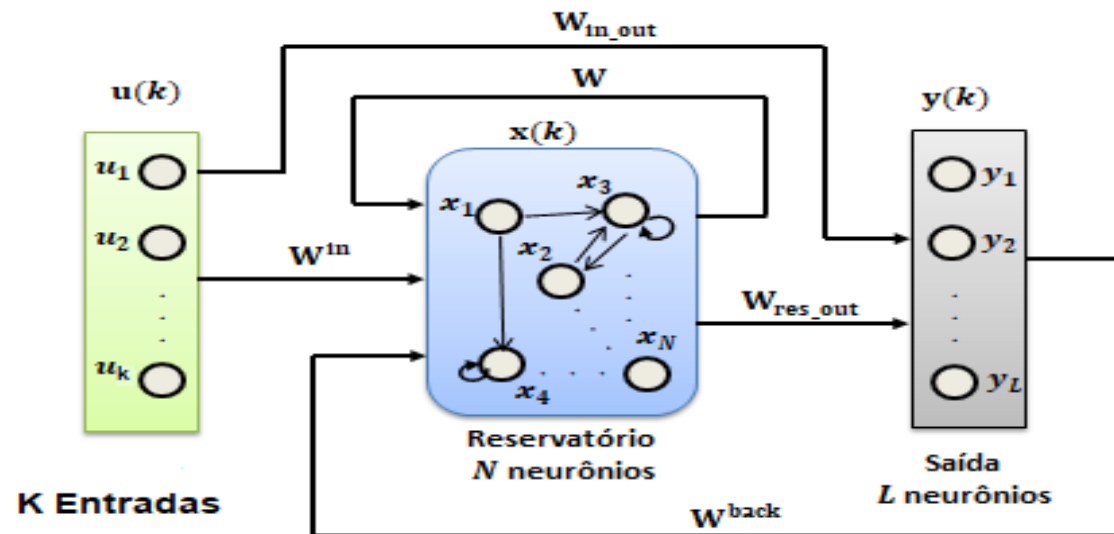
$$x(n+1) = f(W_{in} u(n+1) + Wx(n) + W_{back} y(n))$$

$$y(n+1) = x(n+1)W^{out}(n) + u(n+1)W^{in_out}$$

- Onde, W^{out} é a matriz de pesos de saída após o treinamento, $f(\cdot)$ é a função de ativação do neurônio e $x(n+1)$ é o estado do 'reservoir'.
- O método mais comum para obter os valores de W^{out} é a regressão linear.
- Como somente os **pesos da camada de saída são atualizados** durante o treinamento, a tarefa de **aprendizado da rede se torna simples** (Delta Rule).

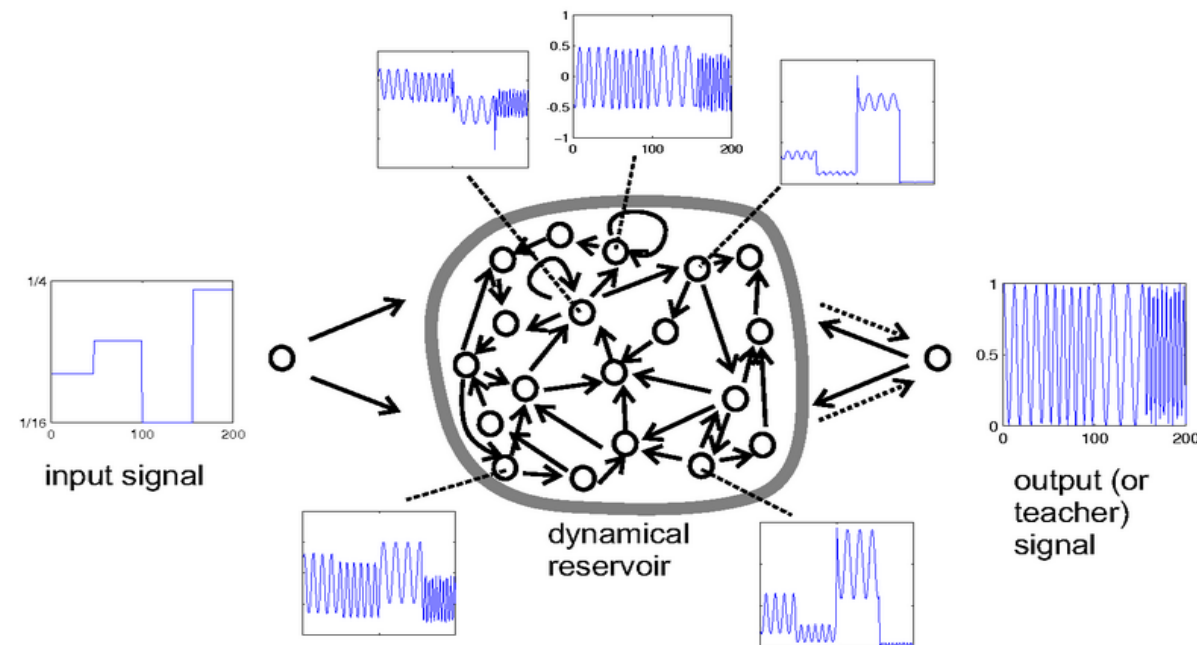
ESNs: Arquitetura - Reservoir

- A parte não treinada da RNN é conhecida como *dynamical reservoir* e os estados resultantes $x(n)$ são chamados *echoes* do histórico de entrada.
- Os pesos criados de maneira aleatória e esparsos permanecem inalterados durante o treinamento.
- As funções de ativação são não lineares.



ESNs: Arquitetura - Reservoir

- › O **reservatório** (*reservoir*) tem como função **gerar** um conjunto de **comportamentos dinâmicos** que seja o mais diversificado possível
- › O **número de neurônios** nesta camada deve ser **grande** e a sua matriz de pesos **W** deve ser **esparsa**.
 - da ordem de 50 a 1000 neurônios



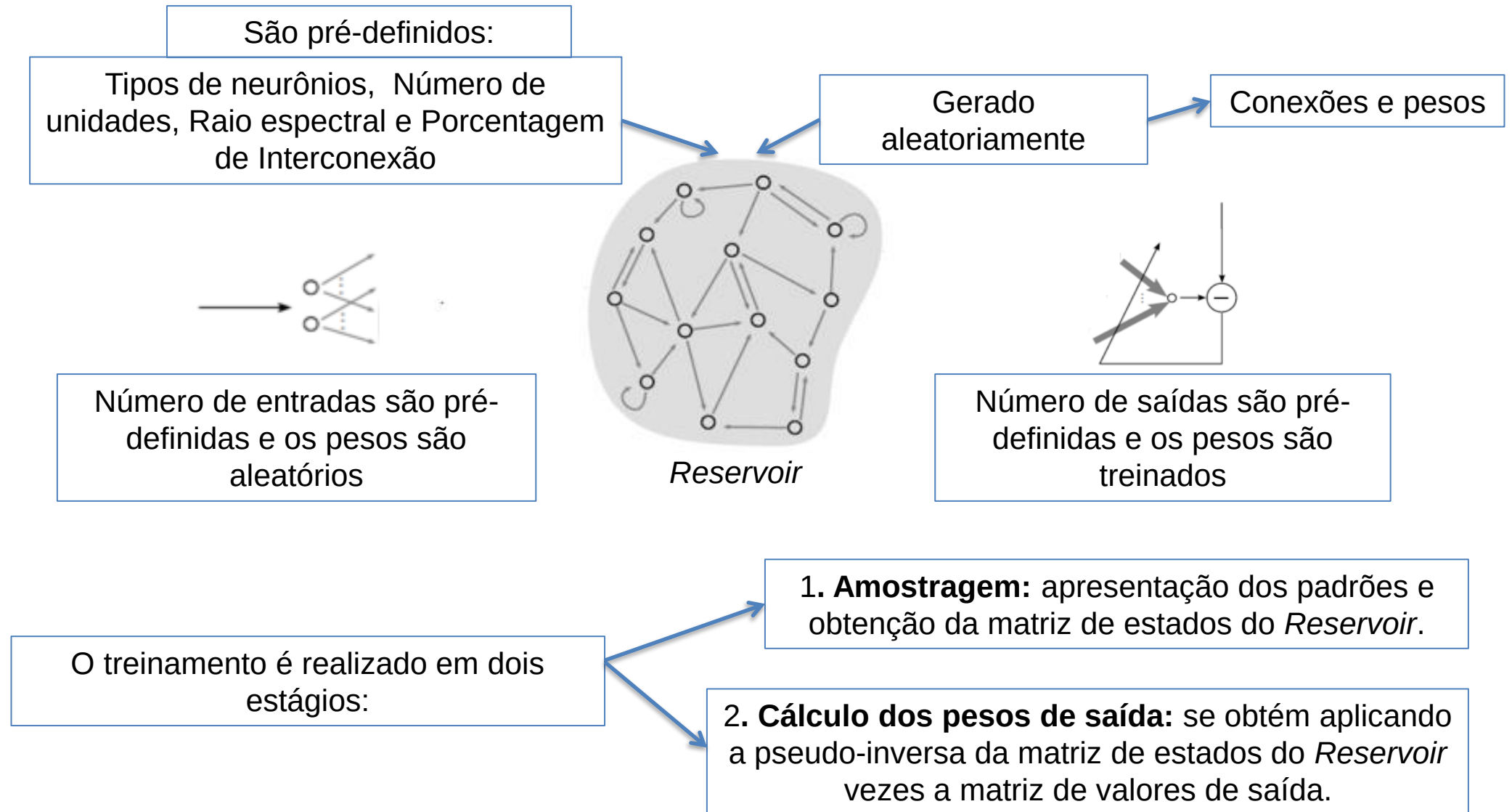
ESNs: Arquitetura - Parâmetros

- › Pesos sinápticos das conexões de entrada e recorrentes são definidos de maneira prévia e aleatória
- › Camada de saída, também chamada de camada de leitura (em inglês, *readout*), produz as saídas da rede por meio de combinações lineares das ativações do reservatório
- › Treinamento resume-se ao ajuste de pesos da camada de saída, o qual pode ser realizado por meio de qualquer método que permita a solução de um problema de mínimos quadrados (Delta Rule)

Echo State Neural Networks (ESN)

- › Introdução
- › Arquitetura
- › Funcionamento Geral
- › Treinamento
- › Observações
- › Aplicações:
 - › Previsão de Séries Temporais
 - › Identificação de Sistemas

ESNs: Funcionamento Geral



ESNs: Funcionamento Geral

- › A expansão não-linear com memória gera um vetor de estado da forma:

$$x(n) = f^{res}(W^{in}u(n) + Wx(n-1)), n = 1, \dots, T$$

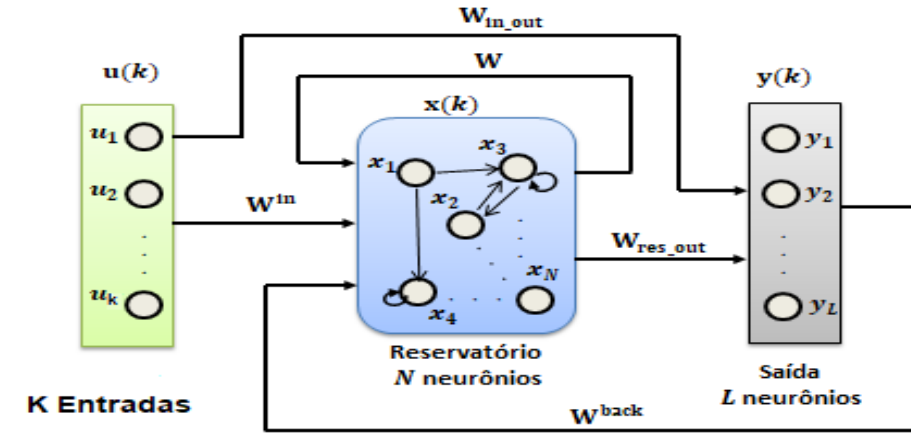
Onde:

$u(n)$ é o vetor das ativações dos neurônios do *Reservoir* no estado n ,

W^{in} é a matriz com os pesos das conexões de entrada e W é matriz de pesos das conexões internas da rede.

Alguns modelos de RNNs **adicionam** W^{back} que é a matriz com os pesos das conexões de retroalimentação da camada de saída.

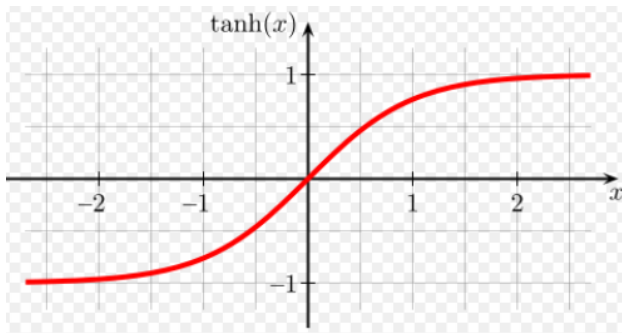
$$x(n) = f^{res}(W^{in}u(n) + Wx(n-1) + W^{back}y(n-1))$$



ESNs: Funcionamento Geral

› As funções de ativação mais usadas nos neurônios do reservatório são:

1. Tangente hiperbólica (*tanh*)



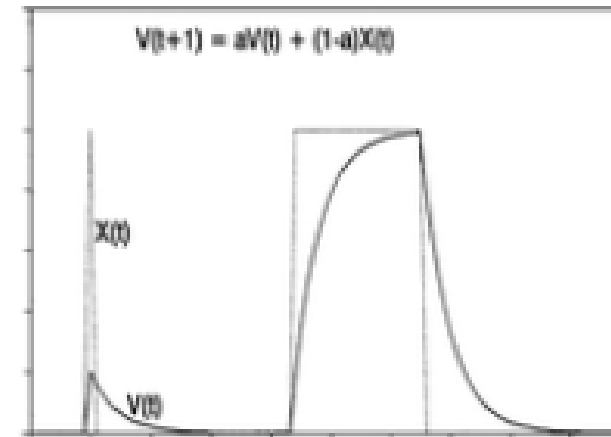
Leaking rate

$$x(n) = (1 - \alpha)x(n - 1) + \alpha f^{res}(W^{in}u(n) + Wx(n - 1) + W^{back}y(n - 1))$$

› $\alpha \in (0,1]$ é a taxa de decaimento, mais conhecida como *leakage rate*.

› valor de α controla a dinâmica de atualização da rede

2. Leaky integrator

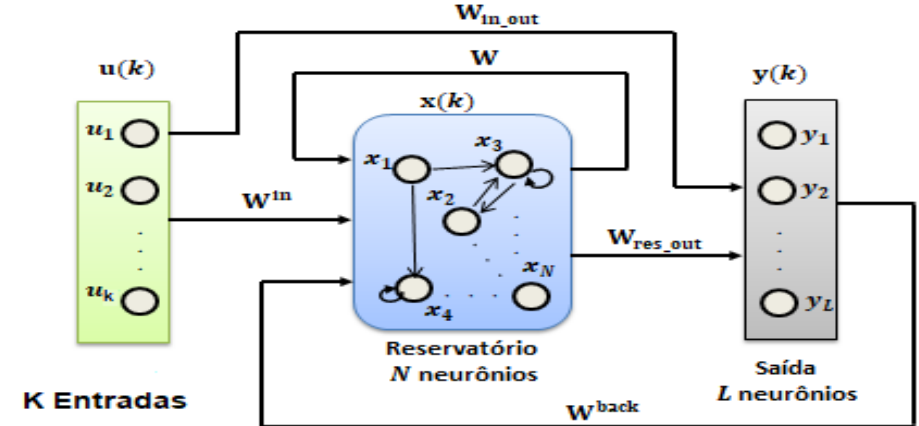


ESNs: Funcionamento Geral

- Saídas

- $y(n) = [y_1(n) \dots y_L(n)]^T$

Os parâmetros da **camada recorrente** não são **ajustados** durante o processo de adaptação da rede, o que significa que apenas os **pesos** da **camada de saída** são **efetivamente ajustados** com base em um **sinal de referência**.



$$\mathbf{y}(n + 1) = \mathbf{f}^{out}(\mathbf{W}^{out}(\mathbf{u}(n + 1), \mathbf{x}(n + 1), \mathbf{y}(n)))$$

devido ao **caráter linear da camada de saída**, os **pesos** ótimos podem ser **determinados** por **mínimos quadrados** com o auxílio de métodos de **regressão linear**. $\Rightarrow$ convergência garantida, um único mínimo global.

ESNs: Funcionamento Geral

- › O **treinamento off-line** de uma ESN é, portanto, realizado por pseudoinversa, por possuir menor custo computacional e ser mais simples.
- › O objetivo do **treinamento** é determinar os **pesos** de saída **W^{out}** de forma a minimizar o erro médio quadrático (MSE – *Mean Squared Error*) entre o vetor de saídas desejadas $y_d(n)$ e o vetor de saídas da rede $y(n)$.

ESN: Funcionamento Geral

- › As matrizes W_{in} , W_{back} e W são geradas aleatoriamente de acordo com alguns parâmetros.
- › Os parâmetros globais que precisam ser definidos são:

- › tamanho do reservatório N
- › esparsidade de W (percentual de conexões no *Reservoir*)
- › raio espectral (ρ_w) de $W \Rightarrow$ Valor de escala dos pesos do reservoir < 1.0 , para que a rede convirja e não apresente um comportamento caótico.
- › fator de escala nos pesos das conexões de entrada - sW^{in}
- › *Leaking rate* $\Rightarrow$ a velocidade da dinâmica de atualização

$$\begin{aligned}\tilde{\mathbf{x}}(n) &= \tanh(\mathbf{W}^{in}[1; \mathbf{u}(n)] + \mathbf{W}\mathbf{x}(n-1)), \\ \mathbf{x}(n) &= (1 - \alpha)\mathbf{x}(n-1) + \alpha\tilde{\mathbf{x}}(n),\end{aligned}$$

- Tamanho (# Unidades): quantidade de unidades (neurônios) do '*reservoir*'.
 - Quanto maior o *reservoir*, mais fácil é de se obter bom desempenho, desde que medidas apropriadas de regularizações sejam tomadas para evitar o superajustamento.
 - Quanto maior o espaço dos sinais $x(n)$ do *reservoir*, mais fácil é a busca por uma combinação linear dos sinais para aproximar $y_{\text{target}}(n)$.

ESNs: Esparsidade de W

- Parâmetro que define a quantidade de células da matriz W iguais a zero.
- Indica a **porcentagem** de **conexões** do '*reservoir*', normalmente para garantir uma dinâmica variada
- Em geral **20%** de **conexões** é um número bom para **iniciar** a **investigação**, aumentando progressivamente.

ESNs: Esparsidade de W

- › Um padrão de conexões **esparsas** tende a favorecer o **desacoplamento** de grupos de **neurônios do reservatório**, contribuindo para o desenvolvimento de **dinâmicas** pouco **correlacionadas**.
- › A **propriedade de estados de eco** garante que a ativação de cada **neurônio** do reservatório torna-se uma **transformação não-linear** do histórico recente do sinal de entrada (por isso o termo eco).

ESNs: Raio Espectral

- Valor ao qual são escalados ou normalizados os pesos, normalmente **menor que 1** para a rede **convergir** e **não apresentar** um **comportamento caótico**.
- Pode ser **interpretado** como o determinante do quão rápido a **influência** de uma **entrada** deixa de **existir** no **reservoir com o tempo**.
- A matriz de pesos W é gerada aleatoriamente e computa-se o maior auto-valor absoluto $= \rho(W_0)$
 - ⇒ divide-se W por $\rho(W_0)$ para obter uma matriz com o raio espectral unitário
 - ⇒ essa nova matriz de pesos é escalada com o raio espectral ρ_w pré-estabelecido.

ESNs: Taxa de Vazamento (*Leaking rate*)

- A Taxa de Vazamento α pode ser interpretada como um regulador da atualização da dinâmica do sistema, ou ainda como a dependência do estado atual com o estado anterior.
- Esse parâmetro governará o quanto do estado anterior o estado atual vai resgatar.

$$\tilde{\mathbf{x}}(n) = \tanh(W^{entrada}[1; \mathbf{u}(n)] + \mathbf{W}\mathbf{x}(n-1))$$

$$\mathbf{x}(n) = (1 - \alpha)\mathbf{x}(n-1) + \alpha\tilde{\mathbf{x}}(n),$$

ESN: Propriedade de *echo state*

- › Após a ESN ter sido executada por um longo período de tempo, os **estados** de **ativação** dos **neurônios** do **reservatório** em um passo de tempo k ($\mathbf{x}(k)$) **dependerão** somente do **histórico** das **entradas** e **saídas** anteriores da **rede** (Jaeger, 2013).

$$\mathbf{x}(n + 1) = \mathbf{f}(\mathbf{W}^{\text{in}}\mathbf{u}(n + 1) + \mathbf{W}\mathbf{x}(n) + \mathbf{W}^{\text{back}}\mathbf{y}(n))$$

- › Esta condição indica que o efeito de um **estado anterior** ($\mathbf{x}(n)$) e de uma **entrada anterior** ($\mathbf{u}(n)$) em um **estado futuro** ($\mathbf{x}(n+k)$), deve desaparecer gradualmente à medida que o tempo passa (ou seja, $k \rightarrow \infty$), e não deve persistir ou mesmo ser amplificado.
- › A existência de tal efeito depende de propriedades algébricas da matriz $\mathbf{W}$.
- › Uma condição prática é fazer com que o raio espectral de $\mathbf{W}$ não exceda uma unidade ($\rho_{\mathbf{W}} < 1$).

Propriedade de *echo state*

- › A propriedade de *echo state* ($\rho_w < 1$) tem levado a um equívoco que aparece em muitos trabalhos de RC, ou seja, que seria uma **condição necessária e suficiente** para a **propriedade de *echo state***.
- › Contrariamente a esse equívoco, a **propriedade de *echo state* pode ser obtida** mesmo que $\rho_w > 1$ em sistemas com entrada diferente de zero, e ela também pode ser perdida mesmo se $\rho_w < 1$, embora seja muito difícil construir sistemas onde isso ocorra.

Propriedade de *echo state*

- › Na prática:
 - › o valor ótimo de ρ_w pode ser escolhido de acordo com a necessidade de memória e de não-linearidade da tarefa.
 - › A regra introduzida por Jaeger é que ρ_w deve ser **próximo** de **1** para tarefas que exigem **muita memória** e, conseqüentemente, **menor** para as **tarefas** onde uma **memória** muito **longa** poderia ser **prejudicial**.

Echo State Neural Networks (ESN)

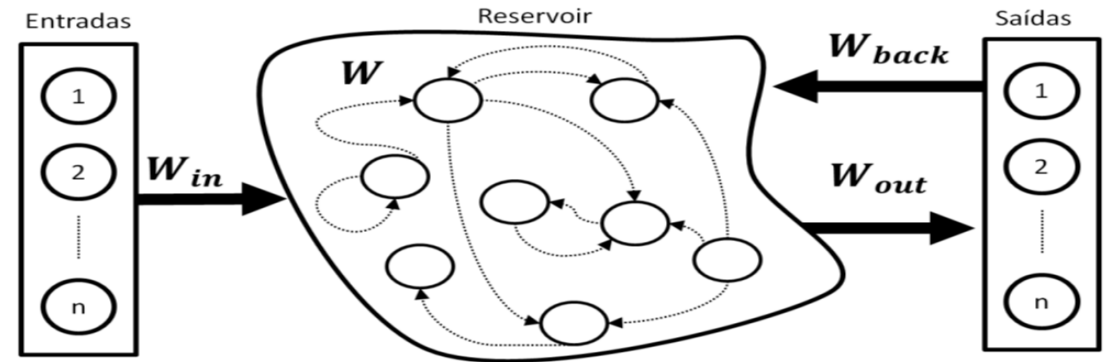
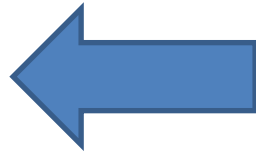
- › Introdução
- › Arquitetura
- › Funcionamento Geral
- › **Treinamento**
- › Observações
- › Aplicações:
 - › Previsão de Séries Temporais
 - › Identificação de Sistemas

ESNs: Treinamento Supervisionado

› As redes ESN têm dois tipos de treinamento:

› *Off-line*

› *On-line*



- Os dois tipos de **treinamento** são feitos para **adaptar** os **pesos** da **camada** de **saída** que conecta os neurônios do *Reservoir* aos neurônios da camada de saída da rede.

ESNs: O treinamento *Off-line*

- Passo 1:

- Geram-se as três matrizes de pesos: W_{in} , W e W_{back}
- $\mathbf{W}^{in} = s_{in} \mathbf{W}_0^{in}$
- $\mathbf{W}^{back} = s_{back} \mathbf{W}_0^{back}$
- $W = \rho_W \frac{W_0}{\rho(W_0)}$
- s_{in} , s_{back} são fatores de escala de entrada e de realimentação
- $\rho(\mathbf{W}_0)$ é o raio espectral (autovalor de maior valor absoluto) de $\mathbf{W}_0$
- ρ_W é um fator multiplicativo que resultará no raio espectral da matriz $\mathbf{W}$.
- A divisão $\mathbf{W}_0/\rho(\mathbf{W}_0)$ resulta numa matriz de raio espectral unitário, ao ser multiplicada por ρ_W gera a matriz $\mathbf{W}$ com raio espectral. De forma a gerar uma ESN com a propriedade de *echo state*, faz-se $\rho_W < 1$.

ESNs: O treinamento *Off-line*

- Passo 2: Definições das duas condições iniciais:

- Na primeira condição define-se o estado inicial do *Reservoir* como $x_0=0$, ou com valores aleatórios (nesse caso tem que considerar que o eco produzido por valores aleatórios vai demorar mais para desaparecer);

$$x(n) = (1 - \alpha)x(n - 1) + \alpha f^{res}(W^{in}u(n) + Wx(n - 1) + W^{back}y(n - 1))$$

- A segunda condição refere-se ao estado inicial da saída, podendo ser $y_0=0$, ou também um valor aleatório; se $W^{back} \neq 0$, deve-se ter a mesma precaução da primeira consideração.

O treinamento *Off-line*

- Passo 3:
 - Calcula-se o estado do *Reservoir* no tempo $n+1$, usando a equação:

$$x(n + 1) = f(W_{in}u(n + 1) + Wx(n) + W_{back}y(n))$$

$$n=0 \rightarrow x(0)=0 \text{ e } y(0)=0 \text{ e } y(n)=y_d(n)$$

$$x(1)=f(W_{in} u(1) + W x(0) + W_{back} y(0))$$

$$x(1)=f(W_{in}u(1))$$

ESNs: O treinamento *Off-line*

- › Passo 4: Para cada $t \geq T_0$ (aquecimento), armazene os dados concatenando:
(entrada/reservatório/saída)
 - › $M = [u(n)^T \ x(n)^T]^T$ e
 - › $D = y_d(n)$
- › O período de *washout* T_0 é utilizado para que o transiente inicial devido à inicialização aleatória dos estados do reservatório não seja considerado durante o cálculo de **Wout**.

ESNs: Aquecimento (número de ciclos do reservatório)

- Dependendo do tamanho da rede e da escala de tempo intrínseca, os intervalos são:
 - › $10 < T_0 < 500$
 - › 10 (para redes pequenas e rápidas)
 - › 500 (para redes grandes e lentas).

ESNs: O treinamento *Off-line*

- Passo 5:
 - Cálculo da matriz de pesos de saída W_{out} , baseado na equação:

$$W_{out} = (Pseudoinverse(\mathbf{M}) * \mathbf{D})^T$$

- A rede (W_{in} , W , W_{back} , W_{out}) está agora pronta a ser utilizada.
- Novas saídas a partir de novas sequências de entrada $u(n)$, são calculadas usando as equações de atualização:

$$\mathbf{x}(n + 1) = \mathbf{f}(\mathbf{W}^{in} \mathbf{u}(n + 1) + \mathbf{W} \mathbf{x}(n) + \mathbf{W}^{back} \mathbf{y}(n)),$$

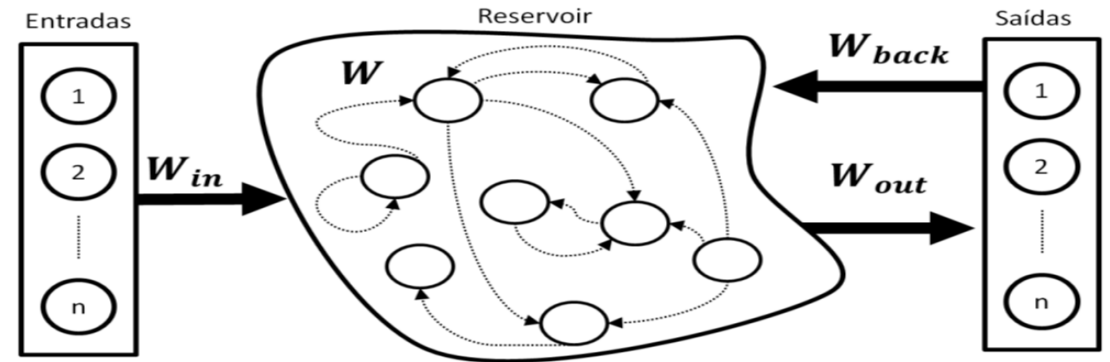
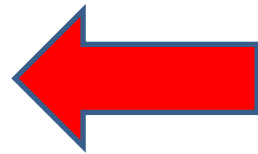
$$\mathbf{y}(n + 1) = \mathbf{f}^{out}(\mathbf{W}^{out}(\mathbf{u}(n + 1), \mathbf{x}(n + 1), \mathbf{y}(n))).$$

ESNs: Treinamento Supervisionado

› As redes ESN têm dois tipos de treinamento:

› *Off-line*

› *On-line*



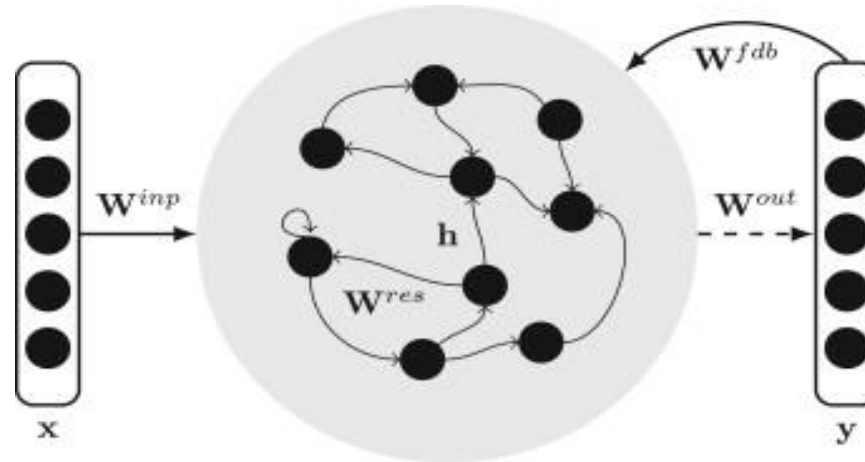
- Os dois tipos de **treinamento** são feitos para **adaptar** os **pesos** da **camada** de **saída** que conecta os neurônios do *Reservoir* aos neurônios da camada de saída da rede.

ESNs: O treinamento *On-line*

1. Passo

› Gerar aleatoriamente as 4 matrizes de pesos:

- › W_{in}
- › $W_{reservoir}$
- › W_{out}
- › W_{back}



Escalar a matriz $W_{reservoir}$, multiplicando-a pelo radio espectral, e com um determinado percentagem de conexão.

2. Passo

Considerações iniciais:

- › Inicializar o estado do reservoir no tempo zero:
 - $X(0) = 0$ ou $\text{Rand}(0-1)$;
- › Inicializar a saída no tempo zero:
 - $y(0) = 0$ ou $\text{Rand}(0-1)$;

$$x(n+1) = \tanh(u(n+1)W^{\text{in}} + x(n)W + y(n)W^{\text{back}})$$

3. Passo

- › Calcular a saída da rede, computando o estado atual do reservoir, com a matriz de pesos de saída.

$$y(n+1) = x(n+1)W^{\text{out}}(n)$$

4. Passo

- › Calcular o erro:

$$e_y = y(n+1) - \hat{y}(n+1)$$

- › Atualizar os pesos (η é a taxa de aprendizado e γ é o momentum)

$$W^{\text{out}}(n+1) = W^{\text{out}}(n) + \eta x(n+1)^T e_y(n+1) + \gamma x(n)^T e_y(n)$$

Echo State Neural Networks (ESN)

- › Introdução
- › Arquitetura
- › Funcionamento Geral
- › Treinamento
- › Observações
- › Aplicações:
 - › Previsão de Séries Temporais
 - › Identificação de Sistemas

ESNs: Observações

- › A matriz W_0 deve ser esparsa, que é um método simples para encorajar uma variedade rica de dinâmicas nas unidades internas.
- › Além disso, os pesos devem ser aproximadamente equilibrados, isto é, o **valor médio** dos **pesos** deve estar em **torno** de **zero**
 - › Pode-se atribuir pesos diferentes de zero a partir de uma distribuição uniforme sobre $[-1, 1]$,
 - › Ou definir pesos não nulos aleatoriamente como -1 ou 1.

ESNs: Observações

- › O tamanho N de W_0 deve refletir tanto o comprimento T dos dados de treino como a dificuldade da tarefa.
- › Como regra geral: $T/10 < N < T/2$
- › Quanto mais periódicos forem os dados de treinamento, mas $N \rightarrow T/2$.
- › Esta é uma simples precaução contra *overfitting*.
- › Além disso, tarefas mais difíceis exigem N maiores.

ESNs: Observações

- › O ajuste do Raio Espectral (ρ_w) é crucial para o desempenho subsequente do modelo.
 - › Para ρ_w :
 - › Pequeno: dinâmica rápida do aprendizado
 - › Grande: dinâmica lenta do aprendizado
- › Tipicamente, ρ_w deve ser ajustado manualmente, testando várias configurações.
- › As configurações padrão: $0,70 < \rho_w < 0,98$

ESNs: Observações

- › O valor absoluto dos pesos de entrada W_{in} também são importantes.
 - › Valores absolutos grandes $\Rightarrow$ a rede é fortemente impulsionada pela entrada.
 - › Valores absolutos pequenos significa que o estado da rede é apenas ligeiramente excitado em torno do repouso (zero).
- › Neste último caso, as unidades da rede operam em torno da parte central linear da sigmóide, isto é, obtém-se uma rede com uma dinâmica linear.
- › Para o W_{in} distribuído uniformemente, geralmente define-se a escala de entrada como “a”, no intervalo do intervalo $[-a; a]$, a partir do qual os valores de W_{in} são amostrados; para W_{in} com distribuição normal, pode usar o desvio padrão como uma medida de escala.

ESNs: Observações

- › **Maiores pesos** da matriz W_{in} dirige as unidades internas para mais perto da **saturação** da **sigmoide**, o que resulta em um **comportamento** mais **não-linear** do modelo resultante.
- › No extremo, quando os **pesos** de W_{in} se tornam muito **grandes**, as unidades internas estarão em um valor quase - 1 / +1 ➡ **dinâmica binária**.
- › Novamente, o **ajuste manual** e **ensaios repetidos** de aprendizagem serão muitas vezes necessários para **encontrar** o **escalonamento adequado** da tarefa.
- › W_{back} similar à matriz W_{in}

ESNs: Observações

- › Muitas vezes, não se deseja ter retorno de saídas.
- › Nesse caso, coletar durante a amostragem apenas os estados da rede retirados os valores das unidades de saída.

$$\mathbf{y}(n + 1) = \mathbf{f}^{out}(\mathbf{W}^{out}(\mathbf{u}(n + 1), \mathbf{x}(n + 1)))$$

ESNs: Observações

- › Se surgirem **problemas** de **estabilidade** ao utilizar a rede, muitas vezes se **adiciona** um **ruído** pequeno durante a amostragem, atualizando a equação:

$$\mathbf{x}(n + 1) = \mathbf{f}(\mathbf{W}^{in} \mathbf{u}(n + 1) + \mathbf{W} \mathbf{x}(n) + \mathbf{W}^{back} \mathbf{d}(n) + \mathbf{v}(n))$$

- › Onde $\mathbf{v}(n)$ é um termo de ruído branco uniforme e pequeno (tamanhos típicos 0,0001 a 0,01)

ESNs: Observações

- › Quando o **sistema** a ser modelado é altamente **não-linear**, melhores modelos são obtidos usando-se estados de rede "aumentados" para treinamento e uso.
- › Isto significa que, **além** dos **estados** de rede **originais** $\mathbf{x}(n)$, as suas **transformações** não **lineares** também devem ser **incluídas**.
- › Um simples caso é incluir os quadrados do estado original: $(\mathbf{x}(n), \mathbf{x}^2(n))$

$$\mathbf{y}(n + 1) = \mathbf{f}^{out}(\mathbf{W}^{out}(\mathbf{u}(n + 1), \mathbf{x}(n + 1), \mathbf{y}(n), \mathbf{u}^2(n + 1), \mathbf{x}^2(n + 1), \mathbf{y}^2(n))).$$

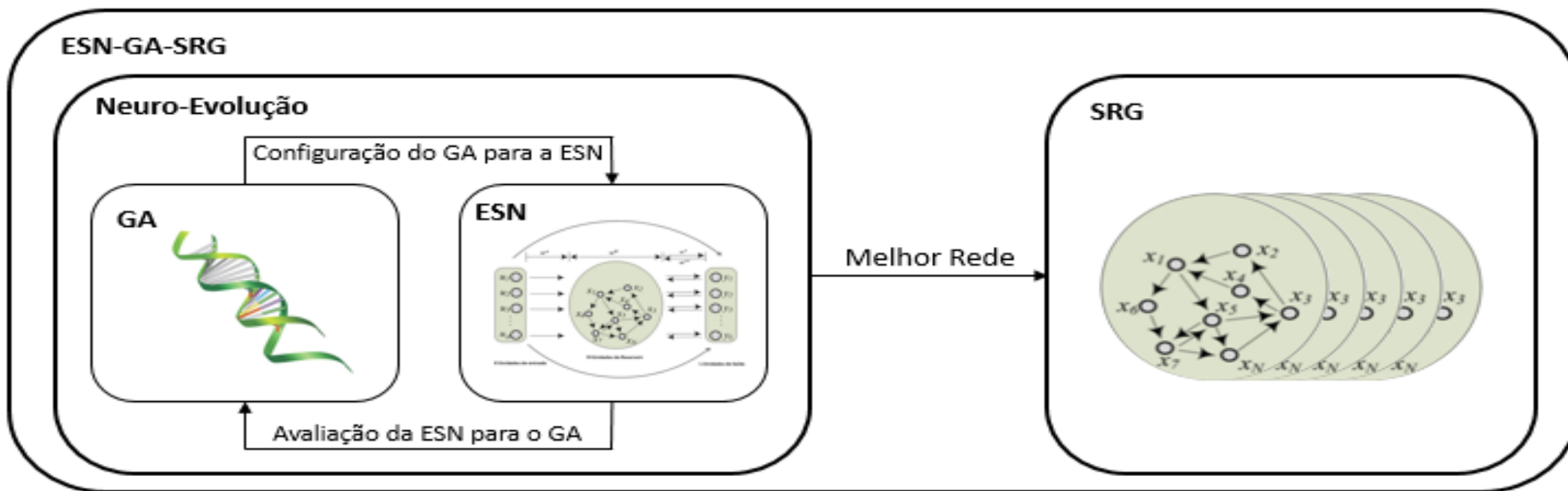
Echo State Neural Networks (ESN)

- › Introdução
- › Arquitetura
- › Funcionamento Geral
- › Treinamento
- › Observações
- › Aplicações:
 - › Previsão de Séries Temporais
 - › Identificação de Sistemas

ESNs: Aplicações – Previsão de Séries Temporais

› Modelo ESN-GA-SRG

- › Uso de Algoritmos Genéticos para determinar os hiper-parâmetros da ESN
- › SRG – Separation Ratio Graph - para escolha do melhor Reservoir



Coefficiente de Separação

- › O coeficiente de separação é a **distância média** entre os **estados** gerados pelo *Reservoir* após a apresentação dos **vetores de entrada** de cada uma das **classes objetivo**.
- › Essa definição se baseia no fato que os **vetores de entrada** e os **estados correspondentes** estão **divididos** em **classes discretas**.
- › Uma vez que **todos** os **estados** do *Reservoir* de uma **mesma classe** têm a **mesma saída**, a **distância** entre **estes estados** deve ser **pequena**.
- › Correspondentemente, estados do *Reservoir* de **diferentes classes** devem ter uma **distância grande**.

A separação do *Reservoir* é especificada em termos de dois fatores: a distância **inter-classe** $C_d(t)$, e a distância **intra-classe** $C_v(t)$

$$SEP_x(t) = \frac{C_d(t)}{C_v(t) + 1}$$

Coeficiente de Separação

- a distância inter-classe $C_d(t)$:

$$C_d(t) = \sum_{m=1}^N \sum_{n=1}^N \frac{||u(O_m(t)) - u(O_n(t))||}{N^2}$$

- a distância intra-classe $C_v(t)$:

$$C_v(t) = \frac{1}{N} \sum_{m=1}^N \frac{\sum_{i=1}^K ||u(O_m(t)) - x_i(t)||}{K}$$

- O centro de massa da classe $O_j \rightarrow u(O_j(t))$:

$$u(O_j(t)) = \frac{\sum_{i=1}^K x_i(t)}{K}$$

$x_i(t)$ representa o estado no tempo t ,

K é o número de estados totais e

$$SEP_x(t) = \frac{C_d(t)}{C_v(t) + 1}$$

Quanto **maior** o **coeficiente de separação**, **maior** a **precisão** na **classificação**, uma vez que a distância inter-classe $C_d(t)$ maior que a distância intra-classe.

Portanto, é possível determinar a qualidade do *Reservoir* **antes** do **treinamento**.

Coefficiente de *Lyapunov*

- › A natureza caótica do *Reservoir* pode ser mensurada empiricamente estimando o coeficiente de *Lyapunov*.
- › Se o valor do coeficiente for positivo, o sistema tem um comportamento caótico e pequenas mudanças na entrada podem apresentar grandes mudanças nos estados do *Reservoir*
- › Um coeficiente de *Lyapunov* negativo resulta em uma rede estável, onde pequenas diferenças na entrada convergem em estados do *Reservoir* que tendem a um atrator.
- › o coeficiente de *Lyapunov*, λ , pode ser calculado para k número de elementos

$$\lambda(t) \sim k \sum_{n=1}^N \ln \left(\frac{\|x_i(t) - x_j(t)\|}{\|u_i(t) - u_j(t)\|} \right)$$

$u_i(t)$ é o vizinho mais próximo de $u_j(t)$
 $x_i(t)$ e $x_j(t)$ são os estados do *Reservoir*,
resultantes dos vetores de entrada $u_i(t)$ e $u_j(t)$.

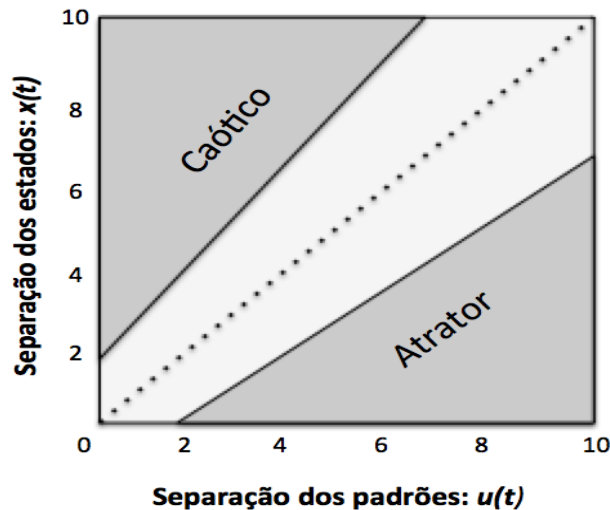
Coeficiente de *Lyapunov*

- › *Reservoir* acontece na “Beira do Caos” onde o coeficiente de Lyapunov está próximo a 0.
- › Em outras palavras, o resultado da razão entre a diferença dos estados e a diferença dos vetores de entrada em um *Reservoir* com resposta ideal deve

$$\frac{||x_i(t) - x_j(t)||}{||u_i(t) - u_j(t)||} \cong 1$$

Separation Ratio Graph

- › Os conceitos de **separação** do *Reservoir* e o **coeficiente** de *Lyapunov* são **unificados** em um método nomeado **Separation Raio** (SR).



Conjunto de vetores de entrada $\{u_1(t), u_2(t), \dots, u_p(t)\}$, e os estados gerados por estas entradas $\{x_1(t), x_2(t), \dots, x_p(t)\}$.

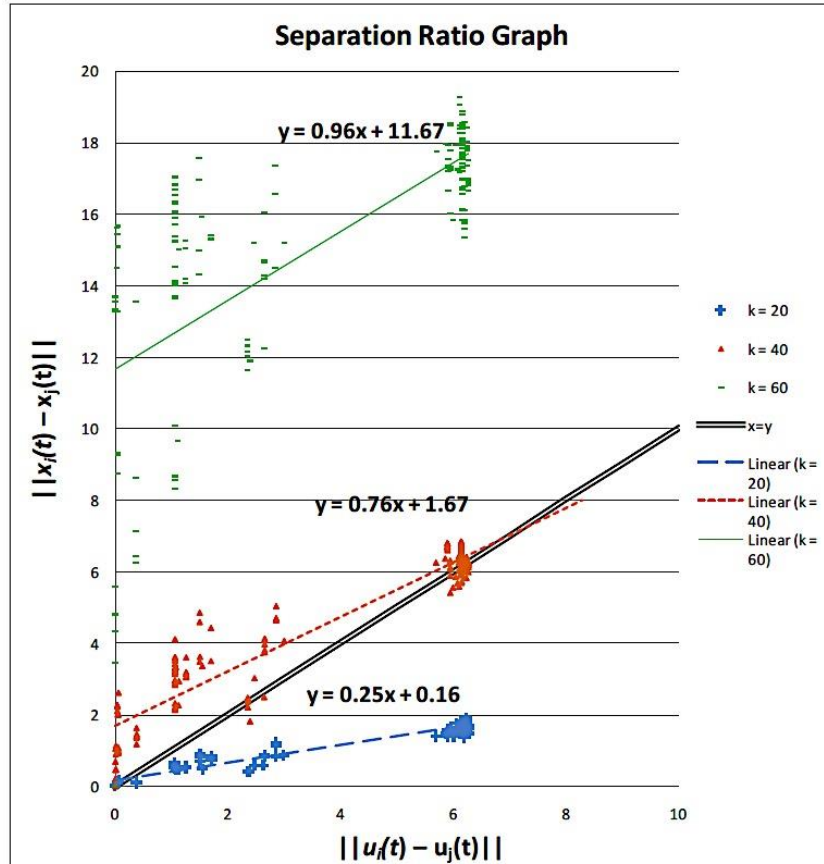
Para cada par de estados, i e $j \in [1, p]$, a distância entre os vetores de entrada $\|u_i(t) - u_j(t)\|$ é apresentada graficamente contra a distância entre os estados correspondentes $\|x_i(t) - x_j(t)\|$

o SR é a razão entre a distância dos vetores de entrada e a distância entre os estados gerados por essas entradas, um *Reservoir* ótimo, $SR \cong 1$

O *Reservoir* apresenta comportamentos indesejáveis em dois casos:

- o comportamento está na parte caótica (entradas similares-respostas diferentes)
- comportamento de um atrator (entradas diferentes-respostas iguais).

Separation Ratio Graph



- › Respostas de 3 redes com comportamentos diferentes do *Reservoir*. Estas redes foram utilizadas para classificação e o parâmetro k , que representa a quantidade de classes, foi modificado em cada uma das experiências realizadas.
- › $k = 20$ é possível observar que a regressão da rede tem um comportamento de um atrator (grandes diferenças na entrada produzem pequenas diferenças na saída);
- › $k = 40$ o resultado obtido na regressão está muito perto do comportamento ideal;
- › $k = 60$ é possível observar uma resposta caótica (pequenas mudanças na entrada geram grandes mudanças na saída)

Separation Ratio Graph

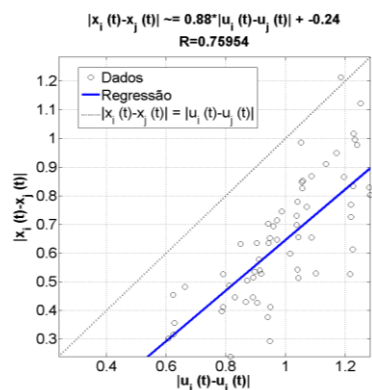
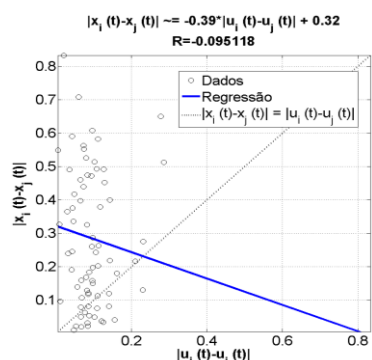
$$SRG = \sum_{p=1}^N |||dif_p(u)|| - ||dif_p(x)||||$$

Onde:

$dif_p(u)$ é a distância entre os dois padrões de entrada que conformam o par p ,

$dif_p(x)$ é a distância entre os dois estados gerados pelos padrões que conformam o par p

N é o número total de pares de padrões gerados pelo algoritmo de agrupamento e vizinhança



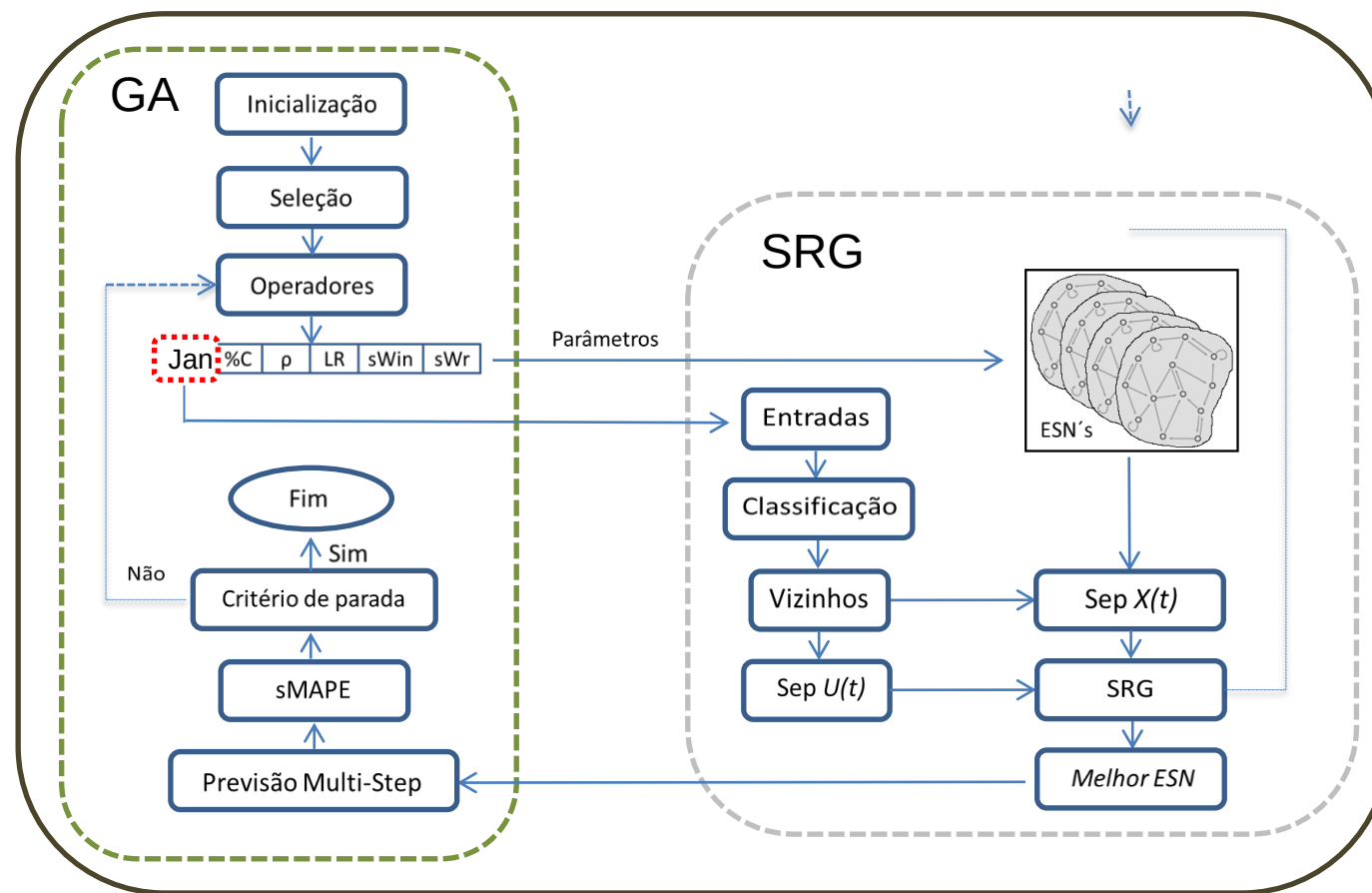
O valor do coeficiente angular não é o melhor (mais próximo de 1), além o ponto de corte do eixo y não é 0, mas, é possível observar que a reta resultante da regressão realizada fica mais perto da linha pontilhada

- › Para interpretar os resultados obtidos por cada *Reservoir*, é realizada uma regressão linear utilizando as distâncias dos vetores de entradas e as distâncias dos estados gerados.
- › A melhor configuração é obtida a partir da reta gerada após a regressão, onde os parâmetros que compõem a equação da reta são avaliados
- › O melhor *Reservoir* é **através** da **regressão** que resulte em m mais próximo 1 e b mais próximo de 0

Benchmark: Conjunto Reduzido NN3

- › Inicialmente foram utilizados algoritmos genéticos para otimização dos parâmetros da ESN:
 - › Janela de entrada.
 - › Percentual de conexões no *reservoir*.
 - › Radio Espectral.
 - › *Leaking rate*.
 - › Escalador das conexões de entrada.
 - › Escalador das conexões recorrentes.

Diagrama de Blocos



Benchmark: Conjunto Reduzido NN3

- › Foi utilizado o método de **Relação Gráfica de Separação** para obter uma análise do comportamento do *reservoir*.

Metodologia:

1. Foi selecionada a melhor configuração obtida de parâmetros para cada série.
2. Foram geradas 100 redes com os parâmetros selecionados no passo anterior, mas com diferentes conexões e pesos no *reservoir*.
3. Foram gerados os conjuntos de dados para cada rede.
4. Utilizando o algoritmo de *k-means* foram separados os registros de entrada em 4 classes.
5. Utilizando a função *knnsearch* foram determinados os vizinhos mais próximos em cada classe.
6. Foram gerados os estados para cada registro de entrada.

Benchmark: Conjunto Reduzido NN3

7. Foi obtida a diferença entre os registros considerados mais próximos e a diferença dos estados gerados pelos mesmos.
8. Com os valores obtidos de cada par de registros e estados, foram calculados:
 - › Regressão Linear.
 - › Coeficiente de Separação.
 - › Coeficiente de Liapunov.
10. Finalmente foram comparados os resultados obtidos com a métrica sMAPE.

Resultado Obtidos

Serie	HW	REG	DEC	NEW	ESN sMAPE_V	ESN sMAPE_T
NN3_101	1,80	2,67	2,61	2,64	2,0004	2,3963
NN3_102	30,63	24,55	29,01	22,05	11,6714	12,5636
NN3_103	29,60	33,81	29,16	34,91	22,5613	25,9191
NN3_104	6,11	9,37	13,78	6,41	7,3278	10,7227
NN3_105	1,53	10,22	10,66	8,28	1,8652	2,8722
NN3_106	5,38	3,62	4,10	6,07	3,3555	3,5192
NN3_107	5,10	3,48	3,29	3,94	3,1879	3,2258
NN3_108	36,29	29,54	29,62	25,83	23,8976	25,5285
NN3_109	7,77	16,88	16,69	8,62	4,4581	5,5109
NN3_110	30,48	27,25	28,62	23,02	21,4520	24,7628
NN3_111	11,14	11,79	11,23	10,49	10,9729	12,2413
Média	15,08	15,74	16,25	13,84	10,2500	11,7511

Echo State Neural Networks (ESN)

- › Introdução
- › Arquitetura
- › Funcionamento Geral
- › Treinamento
- › Observações
- › Aplicações:
 - › Previsão de Séries Temporais
 - › Identificação de Sistemas